

The Inbox tab

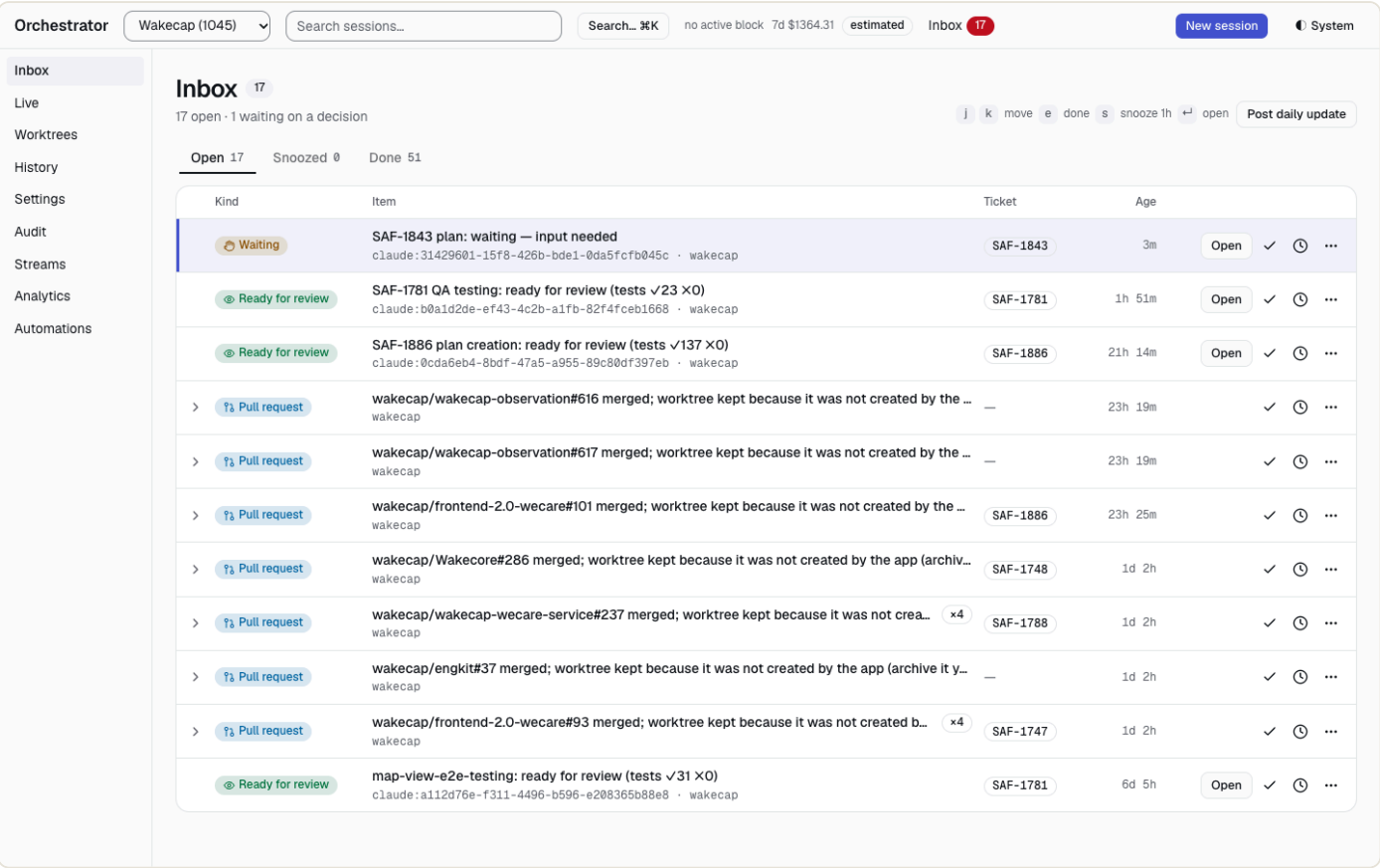
What the `/inbox` page of the orchestrator web UI shows, and the daemon and web code that decides what lands in it, how it is ordered, and how an item leaves it.

Repository: `/Users/hazem/orchestrator` (branch `main`, commit `da4111c`).
Every claim cites `file:line`. Anything the code does not state directly is marked (*inferred*).
Paths under `apps/web/src/` are written `web/...`; under `apps/daemon/src/` as `daemon/...`; under `packages/` as `pkg/...`.

Contents

1. What the user sees
2. Item kinds: who creates them and what each row offers
3. End-to-end data flow
4. The logic: ordering, grouping, states, snooze, keys, actions
5. Sequence: approving a plan
5. Declarations
7. Live data from the running daemon
3. What the code does not settle

1. What the user sees



The Inbox tab at `http://localhost:5173/inbox`, 1440×900, headless Chrome on the running dev server. The project selector is set to Wakecap, so every list is scoped with `projectId`. Browser tab title at capture: `(17) Orchestrator`.

Route and shell

- `/` redirects to `/inbox` in `beforeLoad` – the inbox is the landing page. [web/routes/index.tsx:3–7]
- `/inbox` renders `<InboxPage />`. [web/routes/inbox.tsx:4–8]
- Desktop shell: "Inbox" is the first link in the left nav, and the header carries an `InboxCount` link. [web/features/shell/AppShell.tsx:138, 146–152]
- Phone shell (below 768 px, `MOBILE_QUERY = '(max-width: 767px)'`): the "Orchestrator" title links to `/inbox`, and `InboxCount` sits next to it. [web/features/mobile/useIsMobile.ts:3; AppShell.tsx:88–95]

Page layout, top to bottom

- Heading** "Inbox" with a secondary badge holding the Open count; the badge is hidden when the count is 0 or still loading. [web/features/inbox/InboxPage.tsx:142–143]
- Subtitle.** On the Open tab once loaded without error: `"{n} open · {m} waiting on a decision"`, where `m` counts items of kind `plan_approval` or `waiting`. On every other tab, or while loading or after an error: `"Everything that needs you, newest first."` [InboxPage.tsx:134–135, 145–149]
- Shortcut hint** `j k move · e done · s snooze 1h · ↵ open`, only at the Tailwind `lg` breakpoint and up, and a **"Post daily update"** button (`DailyUpdateButton`). Both are omitted on phones. [InboxPage.tsx:151–168]

4. **Tab** Open / Snoozed / Done, each followed by its item count in mono. [InboxPage.tsx:171–189]
5. **List**. Desktop: a card holding a header row (columns: chevron, Kind, Item, Ticket, Age, actions) over a `<ul aria-label="Inbox items">`. Phone: a stack of `InboxCard` articles. [InboxPage.tsx:215–257, 264]

Tabs and the states they list

Tab	States requested	Empty-state title	Empty-state description
Open	<code>['open']</code>	Nothing needs you right now.	Plan approvals, failing tests, reviews and PR events land here as sessions raise them.
Snoozed	<code>['snoozed']</code>	Nothing snoozed.	Items you snooze wait here until their time comes.
Done	<code>['done', 'auto_resolved']</code>	Nothing done yet.	Items you finish, or that resolve on their own, show here.

[InboxPage.tsx:41–66]

One desktop row (`InboxRow`)

- **Chevron** – only when the item has a detail panel (`hasDetail`: open or snoozed `plan_approval` or `pr_event`); rotates 90° when the row is selected. [InboxPage.tsx:279–299; web/features/inbox/kinds.ts:108–109]
- **Kind** – `KindBadge`: icon plus label, tinted by tone. [web/features/inbox/KindBadge.tsx:6–14]
- **Item** – `reason` (truncated, full text in `title`), a `×N` badge when the row stands for $N \geq 2$ identical items, and a mono sub-line: `source:id · projectId` (or `no project`), plus `· until <local time>` when snoozed. [InboxPage.tsx:303–316; KindBadge.tsx:17–24]
- **Ticket** – outline badge, or an em-dash. [InboxPage.tsx:317–325]
- **Age** – `formatDuration(now – createdAt)`: `45s`, `12m`, `3h 05m`, `2d 4h`. [InboxPage.tsx:326–331; web/features/live-board/sort.ts:72–80]
- **Actions** – `RowActions` (see §4.6). [InboxPage.tsx:332–339]
- Clicking a row selects it; the selected row gets an accent background and a 3 px inset ring on its left edge. A selected row with a detail panel expands `InboxItemActions` under it. [InboxPage.tsx:283–290, 341–345]

One phone card (`InboxCard`)

- Header: kind badge, ticket badge, `×N`, age on the right. Body: full `reason` (wraps), `until ...` line when snoozed, the detail panel when selected. [web/features/inbox/InboxCard.tsx:43–59]
- Buttons for open/snoozed items: **Approve** (only where `offersApprove`), **Done**, **Snooze 1h**, and a "More snooze options" menu with the other two presets. Closed items get **Reopen**. An **"Open session to reply"** link appears when the payload names a session. [InboxCard.tsx:60–101]

Counts, badges and the browser tab title

- **Tab counts.** The page runs three list queries: the two hidden tabs are fetched first only to feed their counts, then the visible tab. A count is `data.length`, so it counts items, not grouped rows. [InboxPage.tsx:80-91]
- **Header `InboxCount`:** a link to `/inbox` with `aria-label="Inbox, {n} open"` and a red (`destructive`) badge when `n > 0`. [web/features/inbox/InboxCount.tsx:4-14]
- **Document title:** `(n) Orchestrator` when `n > 0`, otherwise `Orchestrator`. [web/features/inbox/useInboxTitle.ts:3-11]
- The header count and the title both come from `useOpenInboxCount(projectId)` in `AppShell`, which is the length of the `state: ['open']` list for the selected project. [AppShell.tsx:83-86; web/api/queries/inbox.ts:66-69]
- **Kind badge tones:** `neutral`, `info`, `success`, `warning`, `danger`, mapped to soft fills over the semantic tokens. A `pr_event` whose reason matches `/failed/i` is shown as `danger`. A `waiting` item whose `payload.tool` is a string is labelled "Permission" with a key icon. [kinds.ts:46-80]

Loading and error

- **Loading:** a skeleton – six rows on desktop, four cards on phones – with `role="status"` and `aria-label="Loading inbox"`. [InboxPage.tsx:350-383]
- **Error:** a destructive alert "Couldn't load the inbox" with the error message and a **Retry** button that calls `refetch()`. [InboxPage.tsx:191-202]

2. Item kinds

There are eleven kinds (`InboxKind`, [pkg/core/src/types/inbox.ts:1-12]). The label, icon and tone come from [web/features/inbox/kinds.ts:18-60]. Every kind gets the common row actions in §4.6; this table lists what differs.

Kind · label · tone	Created when (daemon)	Row shows (reason)	Extra actions · leaves when
<code>waiting</code> Waiting (or Permission) warning	A session's live status changes to <code>waiting</code> . <code>statusRule</code> on <code>session.statusChanged</code> , via <code>STATUS_KIND</code> . [daemon/inbox/rules/status-rules.ts:7-11, 53-75]	<code>{name}: waiting – {waitingFor ?? 'input needed'}</code> [status-rules.ts:40-41]	Open, Terminal (owned session). Auto-resolved when the status moves off <code>waiting</code> [status-rules.ts:63], or when the supervisor answers the question [daemon/services/supervisor/supervisor.ts:302].
<code>review</code> Ready for review success	Status changes to <code>review</code> . [status-rules.ts:7-11]	<code>{name}: ready for review (tests ✓P ✗F)</code> [status-rules.ts:42-45]	Open, Terminal. Auto-resolved when status changes to anything except <code>ended</code> : a review item survives the process exiting. [status-rules.ts:61-63]
<code>error</code> Error danger	Status changes to <code>error</code> . [status-rules.ts:7-11]	<code>{name}: API error or crash</code> [status-rules.ts:46-47]	Open, Terminal. Auto-resolved when status leaves <code>error</code> . [status-rules.ts:63]
<code>plan_approval</code> Plan approval info	Status changes to <code>waiting</code> and the transcript holds an <code>ExitPlanMode</code> tool call with no matching <code>tool_result</code> . [daemon/inbox/rules/plan-approval.ts:25-51, 57-75]	Plan awaiting approval. Payload carries <code>plan</code> (redacted, max 20 000 chars) and <code>owned</code> . [plan-approval.ts:11, 41, 71-72]	Detail panel: the plan text, feedback box, Approve plan / Reject plan (confirm dialog). No one-click Approve on the row. Resolved when the session leaves <code>waiting</code> or on approve/reject. [web/features/inbox/PlanApprovalActions.tsx:41-105; kinds.ts:104-105; plan-approval.ts:59-61; daemon/services/review/plan-approval.ts:35, 51]
<code>blocked</code> Blocked warning	No producer found. <code>STATUS_KIND</code> maps only <code>waiting</code> , <code>review</code> , <code>error</code> , and no <code>upsert</code> with <code>kind: 'blocked'</code> exists in <code>apps/daemon/src</code> outside tests. [status-rules.ts:7-11; grep]	–	Notification is on by default for it [daemon/notify/notifier.ts:29-36], so the kind is reserved (<i>inferred</i>).
<code>tests_red</code> Tests red danger	<code>tests.recorded</code> with failures after a previous pass (pass → fail). A fail → fail only refreshes an item that is already active. [status-rules.ts:78-105]	<code>{name}: tests went red (✗F · ✓P)</code> [status-rules.ts:101]	Open, Terminal. Auto-resolved on the next result with <code>failed === 0</code> . [status-rules.ts:84-87]

Kind · label · tone	Created when (daemon)	Row shows (reason)	Extra actions · leaves when
budget Budget warning	Three producers: a configured budget crosses warn or over [daemon/services/usage/budgets.ts:105-141]; the 5-hour block or 7-day quota crosses warnPct [daemon/services/usage/meter.ts:47-69, 137-146]; the monthly LLM recap budget is used up [daemon/services/recap/recap.ts:107-114].	e.g. Over {period} budget for {label}: \$x of \$y, 5h block at 82% ... – resets HH:MM UTC, LLM recap budget used up for ...	Auto-resolved when the alert is no longer raised; after a restart, stale ones are resolved from the key stored in payload . [budgets.ts:142-148; meter.ts:116-128, 148]
automation_result Automation neutral	An automation run reaches a reportable status; notify() is called from four places. [daemon/services/automations/service.ts:159-182, 229, 308, 399, 545]	{automation}: {status} – {first summary line} [service.ts:168]	Approve (row "More" menu on desktop, button on phone) – the daemon marks it done. [kinds.ts:95-105; daemon/services/remote/session-actions.ts:118]
supervisor_escalation Supervisor danger	The supervisor escalates a session's question instead of answering it (not in dry-run). [supervisor.ts:92-130]	Supervisor escalated: {reason} [supervisor.ts:121]	Open. No producer-side resolve found (<i>inferred from grep: no resolve with this kind</i>); leaves by Done.
pr_event Pull request info (danger if reason has "failed")	GitHub poller events: review requested [daemon/inbox/rules/pr-event.ts:31-51]; CI checks failed [pr-event.ts:70-84]; changes requested [pr-event.ts:89-102]. Also: a merged PR's worktree could not be auto-archived [daemon/services/worktree/auto-archive.ts:19-45].	Review requested: ..., CI failed on repo#n: checks, Changes requested on repo#n, repo#n merged; worktree kept because ...	Detail panel: Open PR #n , Review (link to /review/\$source/\$id), and a preset button Fix CI / Address comments when presetId and cwd are set. [web/features/inbox/PrEventActions.tsx:8-48] Auto-resolved when the condition clears or the PR closes. [pr-event.ts:33-35, 61-65, 85-87, 103-105]
reminder Reminder neutral	A user-created reminder's scheduler job fires. [daemon/services/reminders/reminders.ts:30-64, 109-114]	The reminder text, cut to 200 chars. [reminders.ts:61]	Open (when tied to a session). The text may also have been typed into an owned session's terminal. [reminders.ts:37-53]

The row's **Open** button, the Enter key and the phone "Open session to reply" link all need **payload.source** and **payload.id** as strings (**sessionRef**) [kinds.ts:83-86]. The plan-approval rule writes neither – it puts the session pk in **sessionId** [plan-approval.ts:66-72] – so a plan-approval row has no Open button, and Enter on it does nothing. PR-event and budget payloads also carry no source / id .

3. End-to-end data flow

Read top to bottom: events in the daemon become rows in SQLite; the web app reads them over HTTP and keeps them fresh over a WebSocket. User actions go back up over HTTP.

 tinted = runs in the browser (`apps/web`)

untinted = runs in the daemon (`apps/daemon` , `127.0.0.1:4317`)

↓ data toward the screen · ↑ user action toward the daemon

1 • Event sources

who: the daemon process · when: continuously · ends with: an event on `ctx.bus` or a direct `ctx.inbox.upsert()`

live/live-tracker.ts

emits `session.statusChanged` (:420) and `tests.recorded` (:361); never on the first seeding pass (:413–421)

connectors/github/github.ts

emits `pr.changed` (:185) and `pr.reviewRequested` (:194, :203)

direct callers

`usage/budgets.ts` ,
`usage/meter.ts` ,
`recap/recap.ts` ,
`automations/service.ts` ,
`supervisor/supervisor.ts` ,
`reminders/reminders.ts` ,
`worktree/auto-archive.ts`

↓ bus events to registered rules

↑ no upward path at this step

2 • Inbox rules (bus subscribers)

registered: `phase2.ts:54–57` (`status`, `tests_red`) and `main.ts:60–61` (`pr-event`, `plan-approval`) · a throwing rule is logged, not fatal (`engine.ts:242–254`)

inbox/rules/status-rules.ts

`statusRule` , `testsRedRule`

inbox/rules/plan-approval.ts

`createPlanApprovalRule()`

inbox/rules/pr-event.ts

`prEventRule`

↓ `upsert(InboxUpsert)` · `resolve(InboxKey)`

↑ `markDone` · `snooze` · `reopen`

3 • Inbox engine

file: `daemon/inbox/engine.ts` · also runs `tick()` every 15 s to wake snoozed items (:256–270)

inboxDedupeKey()

`inbox/dedupe-key.ts:80–89` –
`kind:scope[:id][:facet]`

db/repos/inbox.ts

SQLite table `inbox_items` ;
unique index on `dedupe_key`
where state is open/snoozed
(`db/schema.ts:163–183`)

ctx.bus.emit

`{ type: 'inbox.upserted',
item }` on every write
(`engine.ts:129`)

notifier.notify(item)

new items and snooze wake-
ups only (`engine.ts:185` ,
`262`)

↓ `list(filter)` · `inbox.upserted`

↑ engine calls from route handlers

4 • HTTP routes and WebSocket

every item passes `redactInboxItem` on the way out (`http/redact-out.ts:259-266`, `http/ws-redact.ts:14-16`)

GET `/api/inbox`

`?state&kind&projectId`
(`http/routes/inbox.ts:18-23`)

POST

`/api/inbox/:id/done|snooze|reopen`
(`http/routes/inbox.ts:25-50`)

POST `/api/inbox/:id/approve`

(`http/routes/session-actions.ts:30-34`)

WS `/ws`

fans out `inbox.upserted`,
sends `hello` on connect
(`http/live-ws.ts:12-27`, `94-95`, `118`)

↓ JSON over HTTP • frames over `/ws` (Vite proxies both)

↑ fetch with `x-orc-token` header

5 • Web data layer

one socket per app, opened by `AppShell` (`AppShell.tsx:80-82`)

`pkg/api-contract client`

`inboxList`, `inboxDone`,
`inboxSnooze`, `inboxReopen`
(`client-p2.ts:86-102`),
`inboxApprove` (`client-p6.ts:44`)

`api/queries/inbox.ts`

`useInbox` → query key
`['inbox', filters]` (:14,
:44-46)

`api/live-events.ts`

`inbox.upserted` →
`upsertInboxItemInCache`
(:116-118); `hello` →
`invalidate ['inbox']` (:93-97)

↓ cached lists re-render

↑ click, key press

6 • Rendering

route `/inbox`

`InboxPage.tsx`

3 queries, `groupItems`, rows
or cards

`AppShell.tsx`

`InboxCount` badge and
`document.title`

`useInboxTriage.ts`

`done` • `snooze` • `reopen` •
`approve`, with toast and
Undo

Step by step

1. **Event.** The live tracker emits `session.statusChanged` only for real transitions after the first pass [`daemon/live/live-tracker.ts:413-421`]. The GitHub connector emits PR events [`daemon/connectors/github/github.ts:185-203`].
2. **Rule.** Each rule subscribes to its bus event types through `registerRule`, wrapped in `try/catch` so one rule's failure is only logged [`daemon/inbox/engine.ts:242-254`].
3. **Upsert.** The engine redacts then truncates `reason` to 300 chars [`engine.ts:20`, 88-90], composes the dedupe key [`engine.ts:147`], and looks for an active row with that key [`engine.ts:148`]. Found: update fields, keep the state, emit, no notification [`engine.ts:149-167`]. Not found: insert a new `open` row with a random UUID, emit, notify [`engine.ts:169-186`].
4. **Storage.** One SQLite row per item; `payload` is stored as JSON text in `payload_json` [`daemon/db/repos/inbox.ts:38-43`; `db/schema.ts:163-183`].

5. **Push.** Every write emits `inbox.upserted` on the bus [engine.ts:129]. The `/ws` hub forwards it to every connected browser after redaction [daemon/http/live-ws.ts:100-118].
6. **Fetch.** `useInbox(filters)` calls `GET /api/inbox?state=...&projectId=...` and validates the response with `InboxItemSchema` [web/api/queries/inbox.ts:44-46; pkg/api-contract/src/client-p2.ts:86-90]. Default `staleTime` is 10 s, `retry` 1; no polling interval is set for the inbox [web/main.tsx:12].
7. **Live patch.** On `inbox.upserted` the web removes the item from every cached `['inbox', ...]` list and re-adds it only where it still matches that list's filters, then re-sorts by `updatedAt` descending [web/api/queries/inbox.ts:28-42]. This is how a row moves between tabs without a refetch.
8. **Reconnect.** The socket retries with backoff `min(30 s, 500 ms × 2^attempt)`; the daemon sends `hello` on each connect, and the web then invalidates all inbox lists and refetches them rather than replaying missed frames [web/api/live-events.ts:93-97, 161-167; daemon/http/live-ws.ts:94-95].
9. **Render.** `InboxPage` groups the list and renders rows or cards [InboxPage.tsx:91, 215-257].
10. **Outside the page.** A notification opens `/sessions/{source}/{id}` when the payload names a session, otherwise `/inbox` [daemon/notify/notifier.ts:43-50]. For web push, the service worker's `notificationclick` focuses an open window and navigates it to that URL, defaulting to `/inbox` [web/sw.ts:36-50]. The service worker never caches `/api` responses [sw.ts:10-13].

4. The logic

4.1 Ordering

- The daemon orders by `updated_at` descending, then `rowid` ascending [daemon/db/repos/inbox.ts:79].
- After a live patch, the web re-sorts each cached list by `updatedAt` descending [web/api/queries/inbox.ts:39].
- The Age column counts from `createdAt` [InboxPage.tsx:330]. A refreshed item (same-key upsert) moves to the top because its `updatedAt` changes [engine.ts:158–160], while its age keeps counting from creation (*inferred from those two lines*).

4.2 Filtering

- Server-side filters: `state` (comma list), `kind` (comma list), `projectId`. Each one that is present becomes one SQL condition, joined with AND [pkg/api-contract/src/routes/inbox.ts:44–50; daemon/db/repos/inbox.ts:70–82].
- The page sends `state` from the tab and `projectId` from the project selector; "all" or no selection sends no project filter [InboxPage.tsx:79–85; web/api/queries/inbox.ts:17–19]. The page never sends `kind`.
- The same filter is re-applied in the browser when a live frame arrives (`matchesInboxFilter`) [web/api/queries/inbox.ts:21–26].

4.3 Grouping duplicates

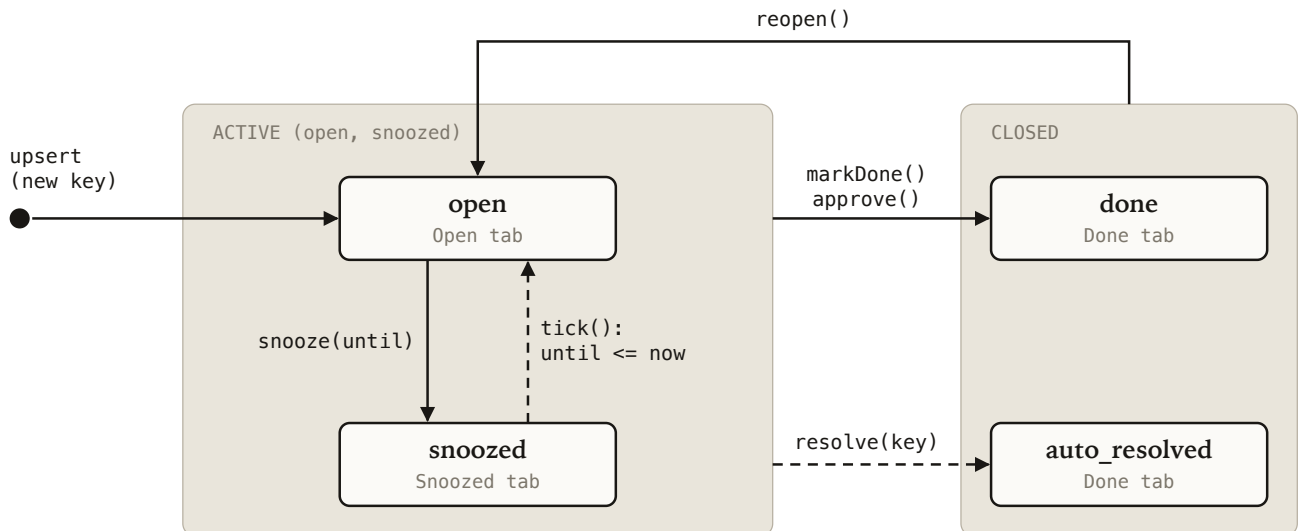
`groupItems` folds items with the same `[kind, reason, ticket]` into one row. The first item seen in a group – the newest, given the order above – is the `lead` whose details the row shows. Groups keep first-appearance order [kinds.ts:111–132]. Done, Snooze and Reopen act on every item in the group; Approve acts on the lead only [web/features/inbox/useInboxTriage.ts:72–93].

```
apps/web/src/features/inbox/kinds.ts:119–132
```

```
/**
 * Collapses items that read the same – one PR notice raised by four sessions, say – into one row.
 * Order follows each group's first appearance, so a newest-first list stays newest-first.
 */
export function groupItems(items: InboxItem[]): InboxGroup[] {
  const groups = new Map<string, InboxGroup>();
  for (const item of items) {
    const key = JSON.stringify([item.kind, item.reason, item.ticket ?? null]);
    const g = groups.get(key);
    if (g) g.items.push(item);
    else groups.set(key, { key, lead: item, items: [item] });
  }
  return [...groups.values()];
}
```

4.4 States and transitions

- solid = caused by a user (or remote) action
- dashed = done by the daemon on its own
- tinted box = a group of states, not a state



Lifecycle of one inbox row. Sources: `engine.ts:144-240` (upsert, resolve, markDone, snooze, reopen) and `engine.ts:256-264` (tick). `approve()` ends in `done` for `automation_result`; for `plan_approval` the plan service resolves the item first, so it usually ends in `auto_resolved` [`services/remote/session-actions.ts:103-118`].

- **Identity.** An item's identity is its dedupe key `{kind}:{scope tag}:{id}[:{facet}]`, each variable part percent-encoded [`daemon/inbox/dedupe-key.ts:74-89`]. The table allows only one open-or-snoozed row per key [`daemon/db/schema.ts:180`].
- **Refresh.** A same-key upsert while the item is active updates `reason`, `updatedAt` and any field the caller passed (including `null`), keeps the state – a snoozed item stays snoozed – and sends no notification [`engine.ts:149-167`].
- **Re-raise after close.** Once an item is done or auto-resolved, the same key no longer matches an active row, so the next upsert inserts a new item [`engine.ts:148, 169-186; db/repos/inbox.ts:50-57`].
- **markDone** sets `done` and clears `snoozeUntil`; it does not check the current state [`engine.ts:203-208`].
- **resolve(key)** sets `auto_resolved` on the active row for that key, if one exists [`engine.ts:189-199`].
- **reopen** returns an already-open item unchanged. If another active row already holds the same key, it returns that row instead of reopening this one [`engine.ts:230-240`].
- Errors: unknown id → `404 not_found`; bad snooze → `400 validation_failed` [`engine.ts:69-78, 137-141, 213-220`].

4.5 Snooze rules and timing

- **Presets**, computed in the browser's local time: `1 hour` (`now + 3 600 000 ms`), `Tomorrow 9:00`, `Next Monday 9:00`. On a Monday, "Next Monday" means seven days later [`web/features/inbox/snooze.ts:1-16`].

- The `s` key and the phone "Snooze 1h" button use the first preset [InboxPage.tsx:122–129; InboxCard.tsx:73–81]. The desktop clock icon opens a menu of all three; the phone "... " menu lists the last two [RowActions.tsx:107–115; InboxCard.tsx:83].
- **Server checks:** `until` must match `ISO_INSTANT` (four-digit year, explicit zone), parse, and be in the future; the item must be open or snoozed. It is stored normalised through `toISOString()` [engine.ts:22–32, 210–228]. A missing `until` is rejected by the route [daemon/http/routes/inbox.ts:43–44].
- **Wake-up:** `tick()` runs once at start and then every 15 000 ms. It reopens every snoozed row with `snooze_until <= now`, emits, and notifies again [engine.ts:256–270; daemon/db/repos/inbox.ts:84–92; daemon/phase2.ts:57]. A snooze therefore ends up to about 15 s after its time (*inferred*).
- **Undo** after Done or Snooze reopens every item in the group, then re-snoozes those that were snoozed with a `snoozeUntil` still in the future [useInboxTriage.ts:39–48].
- Slack can also snooze an item by a number of minutes [daemon/services/remote/slack-bridge.ts:120–124].

4.6 Actions and what each calls

Action (UI)	Shown when	Web call	Daemon
Open	payload names a session [RowActions.tsx:88–92]	navigate to <code>/sessions/\$source/\$id</code> [InboxPage.tsx:105–112]	–
Terminal	that session is live, <code>ownership === 'owned'</code> , not ended [InboxPage.tsx:98–103]	<code>useTerminalStore.open(ptyId, reason)</code> [InboxPage.tsx:252]	–
Done (✓, <code>e</code>)	open or snoozed [kinds.ts:88]	<code>inboxDone(id)</code> per group member [useInboxTriage.ts:72–77]	<code>POST /api/inbox/:id/done → markDone</code> [routes/inbox.ts:41]
Snooze (clock, <code>s</code>)	open or snoozed	<code>inboxSnooze(id, until)</code> per member [useInboxTriage.ts:78–83]	<code>POST /api/inbox/:id/snooze {until} → snooze</code> [routes/inbox.ts:43–45]
Reopen	done or <code>auto_resolved</code> [RowActions.tsx:138–143]	<code>inboxReopen(id)</code> per member [useInboxTriage.ts:84–88]	<code>POST /api/inbox/:id/reopen → reopen</code> [routes/inbox.ts:42]
Approve	<code>automation_result</code> , open or snoozed [kinds.ts:95–105]	<code>inboxApprove(leadId)</code> wrapped in <code>withStepUp</code> [useInboxTriage.ts:27–30, 89–93; web/api/step-up.ts:13–25]	<code>POST /api/inbox/:id/approve {confirm:true} → markDone</code> for <code>automation_result</code> ; 409 <code>not_approvable</code> for kinds other than it and <code>plan_approval</code> [session-actions route :30–34; services/remote/session-actions.ts:84–122]
Copy text	open or snoozed, "More" menu	<code>navigator.clipboard.writeText(reason)</code> [RowActions.tsx:73–80, 131–134]	–

Action (UI)	Shown when	Web call	Daemon
Approve plan / Reject plan	selected plan_approval row with payload.owned === true; otherwise "Resume this session in the app to answer the plan." [PlanApprovalActions.tsx:23, 46-87]	planApprove / planReject via useConfirmedMutation; reject needs non-empty feedback [PlanApprovalActions.tsx:25-38, 70-77]	POST /api/sessions/:source/:id/plan/approve reject; see §5 [daemon/http/routes/plan.ts:20-34]
Open PR #n · Review · Fix CI / Address comments	selected pr_event row with payload.pr / sessionId / presetId + cwd	external link; route /review/\$source/\$id; launchPreset(presetId, ...) [PrEventActions.tsx:21-45]	preset launch is outside the inbox code

Each triage call's response (the updated item) is written into every cached list with `upsertInboxItemInCache` [web/api/queries/inbox.ts:53-64; useInboxTriage.ts:29]. Success shows a toast – "Marked done" or "Snoozed · {label}" with an **Undo** action, "Reopened", "Approved" – and failure shows the error message in a toast [useInboxTriage.ts:50-93]. While any triage mutation is pending, `busy` disables the phone buttons and the Approve menu item [useInboxTriage.ts:67; InboxCard.tsx:64-92; RowActions.tsx:126].

4.7 Keyboard shortcuts

Key	Registry id	Effect
j	inbox-next	select next row (stops at the last)
k	inbox-prev	select previous row (stops at the first)
e	inbox-done	Done on the selected group, if open or snoozed
s	inbox-snooze	Snooze the selected group for 1 hour, if open or snoozed
enter	inbox-open	open the lead item's session, if the payload names one

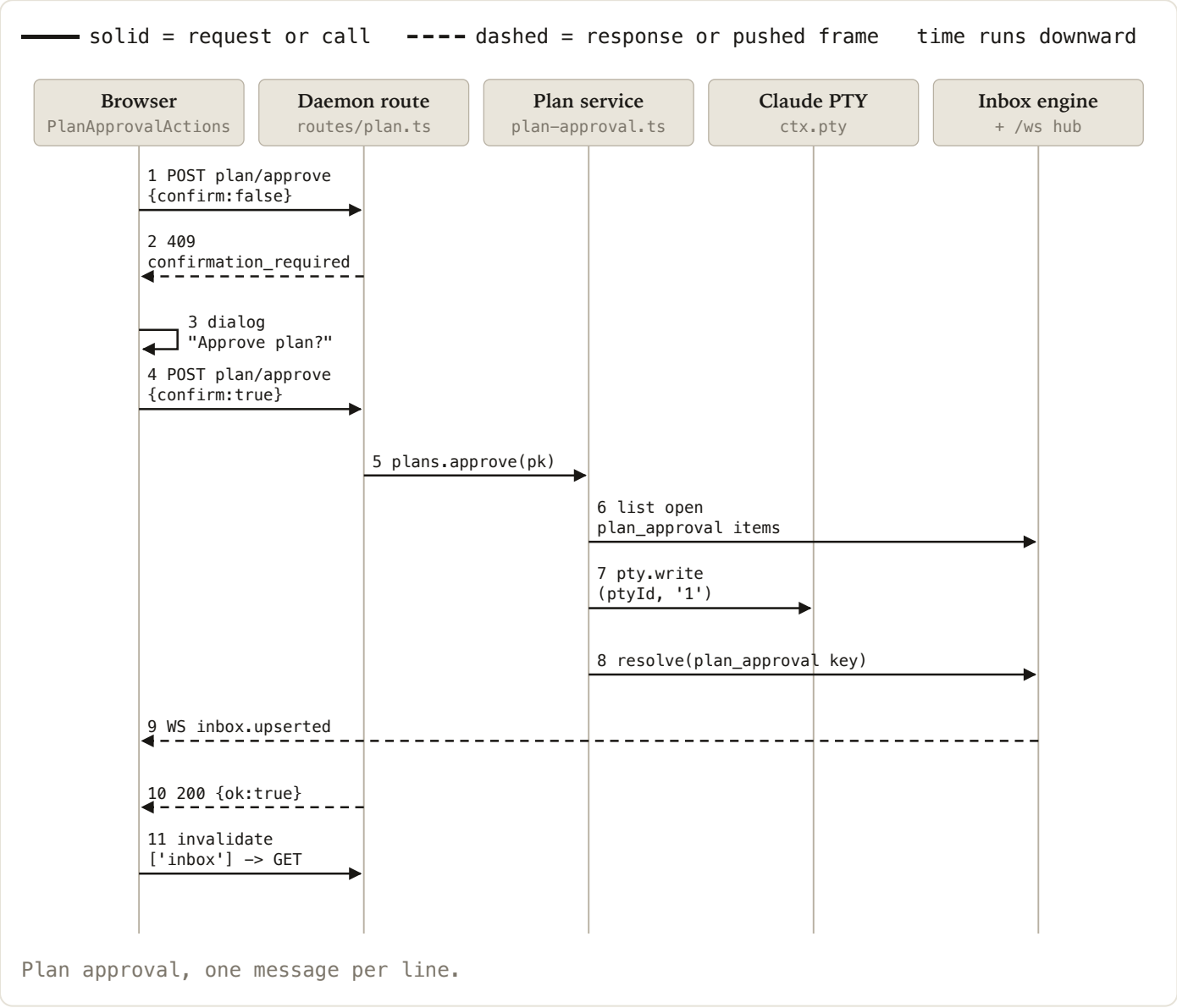
- Registered on the shared hotkey registry in group `inbox` [web/features/inbox/useInboxKeys.ts:28-67]. The registry skips key presses in `INPUT`, `TEXTAREA` and `SELECT` and builds modified keys as `mod+.../alt+...`, so plain bindings do not fire with modifiers [web/features/hotkeys/registry.ts:33, 39].
- Every handler does nothing when the list is empty or the launch dialog is open [useInboxKeys.ts:23-26].
- Selection is clamped when the list shrinks, and resets to 0 on tab change [useInboxKeys.ts:19-21; InboxPage.tsx:173-176].
- Covered by tests: [web/features/inbox/InboxPage.test.tsx:88-129] and the e2e run [apps/web/e2e/live-inbox.spec.ts:65-78].

4.8 How an item leaves a tab

1. **User Done / Approve** → state `done` → leaves Open/Snoozed, appears in Done.
2. **User Snooze** → state `snoozed` → moves Open → Snoozed; `tick()` brings it back to Open when due.
3. **Condition clears** → the producing rule or service calls `resolve(key)` → `auto_resolved` → Done tab (§2 lists each trigger).
4. **Plan answered** → the plan service resolves the plan item; the remote/Slack approve path also marks it done if still active [`daemon/services/review/plan-approval.ts:35, 51;`
`services/remote/session-actions.ts:114-116`].
5. In every case the change reaches the page as an `inbox.upserted` frame, and `upsertInboxItemInCache` drops it from lists whose filter it no longer matches [`web/api/queries/inbox.ts:28-42`].
6. No code path deletes an inbox row (*inferred: `daemon/db/repos/inbox.ts` has no delete function*).

5. Sequence: approving a plan

The plan panel's **Approve plan** button. Who: the user in the browser · when: a `plan_approval` row is selected and its session is owned by the app · ends with: the key `1` typed into the Claude terminal and the item `auto_resolved`.



- `approve.run(null)` sends `planApprove(source, id, {confirm:false})`; `source/id` come from splitting `item.sessionId` [PlanApprovalActions.tsx:14-17, 24-31, 66].
- `requireConfirm` throws `409 confirmation_required` with a summary [daemon/http/routes/plan.ts:23; routes/git-guard.ts:39-45]. Only `claude` sessions are accepted; others get `400` [plan.ts:14-18].
- `useConfirmedMutation` turns that `409` into `pending`, which opens the "Approve plan?" dialog [web/features/git/useConfirmedMutation.ts:45-54; PlanApprovalActions.tsx:88-95].
- Confirm resends with `confirm: true` [useConfirmedMutation.ts:73-77].
- The route calls `plans.approve(pk)` [plan.ts:24].
- `ownedPty` checks that the session exists, is owned, and has an open or snoozed `plan_approval` item with that key; otherwise `not_found`, `not_owned` or `no_pending_plan` [daemon/services/review/plan-approval.ts:17-27].

7. It writes `PLAN_KEYS.approve ('1')` to the PTY inside an audited block `plan.approve` [`plan-approval.ts:30-34`; `services/review/plan-keys.ts:6`].
8. It resolves the plan item → `auto_resolved` [`plan-approval.ts:35`; `engine.ts:189-199`].
9. The engine's emit is pushed to the browser; the row leaves the Open tab [`engine.ts:129`; `web/api/live-events.ts:116-118`].
10. The route answers `{ ok: true }` [`plan.ts:25`].
11. On success the hook invalidates `['inbox']`, so every inbox list refetches [`useConfirmedMutation.ts:43`; `PlanApprovalActions.tsx:12, 30`].

Reject follows the same path with `{feedback, confirm}`; the service writes `Escape`, waits 300 ms, then types a prompt that starts "The plan is not approved yet. Do not start implementing." followed by the feedback [`plan-approval.ts:37-52`; `plan-keys.ts:6-10`]. The inbox-level approve (`POST /api/inbox/:id/approve`, used by remote/Slack callers) calls the same `plans.approve` and then marks the item done if it is still active [`services/remote/session-actions.ts:103-117`].

6. Declarations

6.1 Core types

packages/core/src/types/inbox.ts:1-27

```
export type InboxKind =
  | 'waiting'
  | 'review'
  | 'plan_approval'
  | 'blocked'
  | 'error'
  | 'tests_red'
  | 'budget'
  | 'automation_result'
  | 'supervisor_escalation'
  | 'pr_event'
  | 'reminder';
export type InboxState = 'open' | 'snoozed' | 'done' | 'auto_resolved';
export interface InboxItem {
  id: string;
  kind: InboxKind;
  sessionId: string | null;
  projectId: string | null;
  ticket: string | null;
  reason: string;
  dedupeKey: string;
  createdAt: string;
  updatedAt: string;
  state: InboxState;
  snoozeUntil: string | null;
  payload: Record<string, unknown>;
}
```

6.2 API contract: schemas and bodies

packages/api-contract/src/routes/inbox.ts:1-54

```
import type { InboxItem } from '@orc/core';
import { z } from 'zod';

export const InboxKindSchema = z.enum([
  'waiting',
  'review',
  'plan_approval',
  'blocked',
  'error',
  'tests_red',
  'budget',
  'automation_result',
  'supervisor_escalation',
  'pr_event',
  'reminder',
]);
export type InboxKindSchema = z.output<typeof InboxKindSchema>;

export const InboxStateSchema = z.enum(['open', 'snoozed', 'done', 'auto_resolved']);
export type InboxStateSchema = z.output<typeof InboxStateSchema>;

/** GET /api/inbox response item. `reason` and `payload` may carry transcript-derived text – see task-2-report.md. */
export const InboxItemSchema: z.ZodType<InboxItem> = z.object({
  id: z.string(),
  kind: InboxKindSchema,
  sessionId: z.string().nullable(),
  projectId: z.string().nullable(),
  ticket: z.string().nullable(),
  reason: z.string(),
  dedupeKey: z.string(),
  createdAt: z.string(),
  updatedAt: z.string(),
  state: InboxStateSchema,
  snoozeUntil: z.string().nullable(),
  payload: z.record(z.string(), z.unknown()),
});
```

```

ticket: z.string().nullable(),
reason: z.string(),
dedupeKey: z.string(),
createdAt: z.string(),
updatedAt: z.string(),
state: InboxStateSchema,
snoozeUntil: z.string().nullable(),
payload: z.record(z.string(), z.unknown()),
});

const csv = (s: string): string[] =>
  s
    .split(',')
    .map((x) => x.trim())
    .filter(Boolean);

/** GET /api/inbox?state&kind&projectId - a query schema, so it stays permissive (no `.strict()`). */
export const InboxListQuery = z.object({
  state: z.string().transform(csv).pipe(z.array(InboxStateSchema)).optional(),
  kind: z.string().transform(csv).pipe(z.array(InboxKindSchema)).optional(),
  projectId: z.string().optional(),
});
export type InboxListQuery = z.output<typeof InboxListQuery>;

/** POST /api/inbox/:id/(done|snooze|reopen) body. */
export const InboxActionBody = z.strictObject({ until: z.string().optional() });
export type InboxActionBody = z.output<typeof InboxActionBody>;

```

packages/api-contract/src/routes/plan.ts:1-5

```

import { z } from 'zod';
import { Confirm } from './common.ts';

export const PlanApproveBody = z.object({ confirm: Confirm });
export const PlanRejectBody = z.object({ feedback: z.string().trim().min(1).max(8000), confirm: Confirm });

```

6.3 API contract: client methods

packages/api-contract/src/client-p2.ts:86-102

```

inboxList: (f: { state?: InboxState[]; kind?: InboxKind[]; projectId?: string } = {}) =>
  call<unknown>(
    'GET',
    `/api/inbox${qs({ state: f.state?.join(','), kind: f.kind?.join(','), projectId: f.projectId })}`,
  ).then((d) => z.array(InboxItemSchema).parse(d)) as Promise<InboxItem[]>,
inboxDone: (id: string) =>
  call<unknown>('POST', `/api/inbox/${enc(id)}/done`, { }).then((d) =>
    InboxItemSchema.parse(d),
  ) as Promise<InboxItem>,
inboxSnooze: (id: string, until: string) =>
  call<unknown>('POST', `/api/inbox/${enc(id)}/snooze`, { until }).then((d) =>
    InboxItemSchema.parse(d),
  ) as Promise<InboxItem>,
inboxReopen: (id: string) =>
  call<unknown>('POST', `/api/inbox/${enc(id)}/reopen`, { }).then((d) =>
    InboxItemSchema.parse(d),
  ) as Promise<InboxItem>,

```

packages/api-contract/src/client-p6.ts:44-44

```

inboxApprove: (id: string) => call<InboxItem>('POST', `/api/inbox/${enc(id)}/approve`, { confirm: true }),

```

packages/api-contract/src/client-phase4.ts:137-140

```

planApprove: (source: Source, id: string, body: { confirm: boolean }) =>
  req<{ ok: true }>('POST', `/api/sessions/${source}/${enc(id)}/plan/approve`, { body }),
planReject: (source: Source, id: string, body: { feedback: string; confirm: boolean }) =>
  req<{ ok: true }>('POST', `/api/sessions/${source}/${enc(id)}/plan/reject`, { body }),

```

6.4 Live event union

packages/api-contract/src/live.ts:14–30

```

/** contracts $6 – WS /ws live events (the socket itself ships in Phase 2). */
export type LiveEvent =
  | { type: 'session.updated'; session: Session }
  | { type: 'session.removed'; pk: string }
  | { type: 'inbox.upserted'; item: InboxItem }
  | { type: 'pty.exited'; ptyId: string; code: number | null }
  | { type: 'index.progress'; done: number; total: number }
  | { type: 'usage.updated'; snapshot: UsageSnapshot }
  | { type: 'hello'; serverTime: string }
  | { type: 'audit.recorded'; entry: AuditEntry }
  | { type: 'worktree.updated'; worktree: WorktreeView }
  | { type: 'worktree.removed'; path: string }
  | { type: 'pr.updated'; status: PrStatus }
  | { type: 'checkpoint.created'; checkpoint: CheckpointRecord }
  | { type: 'automation.runUpdated'; run: AutomationRunDetail }
  | { type: 'compare.updated'; group: CompareGroup }
  | { type: 'supervisor.decided'; decision: SupervisorDecisionView };

```

6.5 HTTP routes

Method	Path	Body	Response · registered at
GET	<code>/api/inbox?state&kind&projectId</code>	–	<code>InboxItem[]</code> ; 400 on a bad filter · <code>daemon/http/routes/inbox.ts:18–23</code>
POST	<code>/api/inbox/:id/:action{done snoodle reopen}</code>	<code>InboxActionBody</code> (<code>{until?}</code> , strict)	<code>InboxItem</code> ; 400/404 · <code>daemon/http/routes/inbox.ts:25–50</code>
POST	<code>/api/inbox/:id/approve</code>	<code>ApproveBody</code> (<code>{confirm}</code>)	<code>InboxItem</code> ; 409 <code>confirmation_required</code> / <code>not_approvable</code> · <code>daemon/http/routes/session-actions.ts:30–34</code>
POST	<code>/api/sessions/:source/:id/plan/approve</code>	<code>PlanApproveBody</code>	<code>{ok:true}</code> ; 409 until confirmed · <code>daemon/http/routes/plan.ts:20–26</code> , mounted under <code>/api</code> at <code>daemon/http/app.ts:132</code>
POST	<code>/api/sessions/:source/:id/plan/reject</code>	<code>PlanRejectBody</code>	<code>{ok:true}</code> · <code>daemon/http/routes/plan.ts:28–34</code>
WS	<code>/ws?token=...</code>	–	<code>hello</code> , <code>inbox.upserted</code> , ... · <code>daemon/http/live-ws.ts:12–27, 52</code>

6.6 Daemon engine and dedupe key

apps/daemon/src/inbox/engine.ts:39-67

```
export interface InboxUpsert extends InboxKey {
  sessionId?: string | null;
  projectId?: string | null;
  ticket?: string | null;
  reason: string;
  payload?: Record<string, unknown>;
}

export interface InboxRule {
  name: string;
  on: BusEvent['type'][];
  handle(e: BusEvent, ctx: DaemonContext): void;
}

export interface InboxEngine {
  upsert(item: InboxUpsert): InboxItem;
  resolve(key: InboxKey): void;
  list(filter: { state?: InboxState[]; kind?: InboxKind[]; projectId?: string }): InboxItem[];
  markDone(id: string): InboxItem;
  snooze(id: string, until: string): InboxItem;
  reopen(id: string): InboxItem;
  registerRule(rule: InboxRule): void;
}

export interface InboxEngineRuntime extends InboxEngine {
  tick(now?: Date): void;
  start(intervalMs?: number): void;
  stop(): void;
}
```

apps/daemon/src/inbox/dedupe-key.ts:47-57

```
/** The identity of an inbox item: what it is (`kind`) and what it is about (`scope`, `facet`). */
export interface InboxKey {
  kind: InboxKind;
  scope: InboxScope;
  /**
   * An extra discriminator for the case where one kind raises several *distinct* items for one
   * scope – a PR's `checks` and its `review`, say. Adding a facet can only ever split one key into
   * two; it can never merge two scopes into one, so it cannot reintroduce the suppression bug.
   */
  facet?: string;
}
```

apps/daemon/src/inbox/dedupe-key.ts:25-45

```
export type InboxScope =
  /** A single session, addressed by its pk (`${source}:${id}`). */
  | Exclusive<{ session: string }>
  /** A whole project, addressed by its project id. */
  | Exclusive<{ project: string }>
  /** A ticket, across whatever sessions touch it. */
  | Exclusive<{ ticket: string }>
  /**
   * Anything else that has a stable identity of its own: a PR, a worktree, an automation run, a
   * quota window. `domain` names the namespace and `id` addresses the thing inside it – e.g.
   * `{ domain: 'pr', id: 'owner/repo#4' }`, `{ domain: 'worktree', id: '/Users/x/wt' }`. This is
   * the escape hatch that keeps later phases from falling back on `{ global: true }` plus a
   * hand-composed facet, which would reintroduce the very hand-written key this type removes.
   *
   * `session`, `project` and `ticket` are sugar for the domains of the same name, so
   * `{ domain: 'session', id: pk }` composes to the same key as `{ session: pk }`. That is
   * aliasing of one identity, not a collision of two.
   */
  | Exclusive<{ domain: string; id: string }>
```

```
/** Genuinely global: one item for the whole daemon (use `facet` to have more than one). */  
| Exclusive<{ global: true }>;
```

6.7 Storage

apps/daemon/src/db/schema.ts:163–183

```
export const inboxItems = sqliteTable(  
  'inbox_items',  
  {  
    id: text('id').primaryKey(),  
    kind: text('kind').$type<InboxKind>().notNull(),  
    sessionId: text('session_id'),  
    projectId: text('project_id'),  
    ticket: text('ticket'),  
    reason: text('reason').notNull(),  
    dedupeKey: text('dedupe_key').notNull(),  
    createdAt: text('created_at').notNull(),  
    updatedAt: text('updated_at').notNull(),  
    state: text('state').$type<InboxState>().notNull(),  
    snoozeUntil: text('snooze_until'),  
    payloadJson: text('payload_json').notNull().default('{}'),  
  },  
  (t) => [  
    uniqueIndex('inbox_items_active_dedupe').on(t.dedupeKey).where(sql`state in ('open', 'snoozed')`),  
    index('inbox_items_state_idx').on(t.state, t.updatedAt),  
  ],  
);
```

6.8 Web types and route definitions

apps/web/src/api/queries/inbox.ts:5–14

```
export interface InboxFilters {  
  state?: InboxState[];  
  kind?: InboxKind[];  
  projectId?: string;  
}  
  
/** The prefix every filtered inbox list shares, for invalidation and cache sweeps. */  
export const inboxRootKey = ['inbox'] as const;  
  
export const inboxKey = (f: InboxFilters) => ['inbox', f] as const;
```

apps/web/src/api/queries/inbox.ts:48–51

```
export type InboxAction =  
  | { id: string; action: 'done' }  
  | { id: string; action: 'reopen' }  
  | { id: string; action: 'snooze'; until: string };
```

apps/web/src/features/inbox/useInboxTriage.ts:10–16

```
export interface InboxTriage {  
  done(group: InboxGroup): void;  
  snooze(group: InboxGroup, until: string, label: string): void;  
  reopen(group: InboxGroup): void;  
  approve(group: InboxGroup): void;  
  busy: boolean;  
}
```

apps/web/src/features/inbox/kinds.ts:111-117

```
/** One inbox row: an item, plus any others with the same kind, reason and ticket. */
export interface InboxGroup {
  key: string;
  /** The newest member; the row shows its details and kind panel. */
  lead: InboxItem;
  items: InboxItem[];
}
```

apps/web/src/features/inbox/InboxPage.tsx:33-39

```
type Tab = 'open' | 'snoozed' | 'done';
interface TabDef {
  id: Tab;
  label: string;
  states: InboxState[];
  empty: { title: string; description: string };
}
```

apps/web/src/routes/inbox.tsx:1-8

```
import { createFileRoute } from '@tanstack/react-router';
import { InboxPage } from '@features/inbox/InboxPage.tsx';

export const Route = createFileRoute('/inbox')({ component: InboxRoute });

function InboxRoute() {
  return <InboxPage />;
}
```

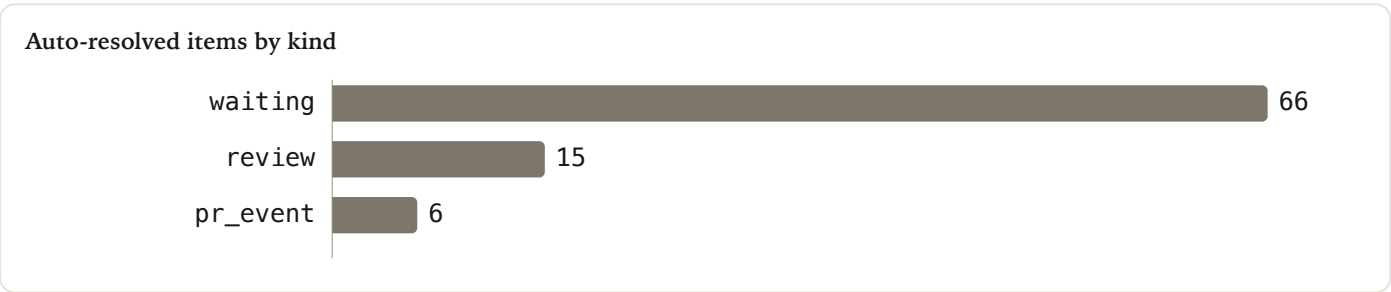
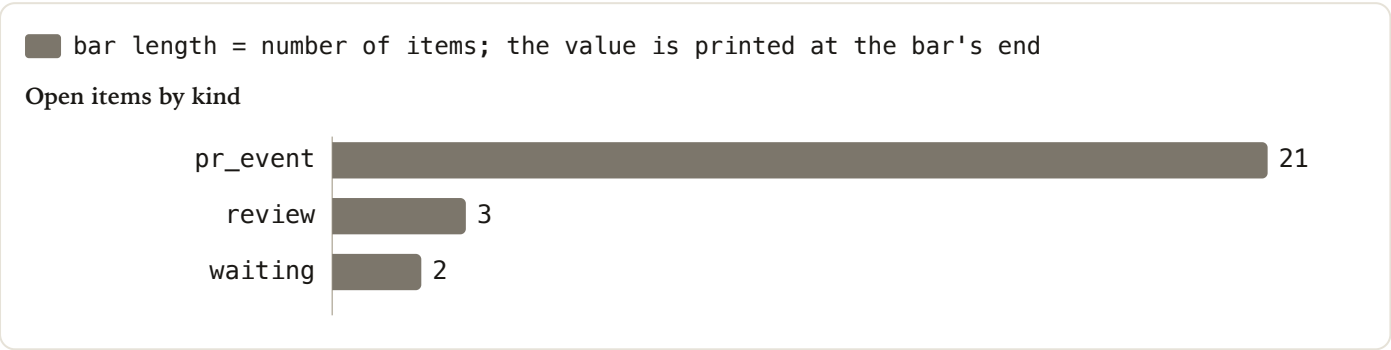
apps/web/src/routes/index.tsx:1-7

```
import { createFileRoute, redirect } from '@tanstack/react-router';

export const Route = createFileRoute('/')({
  beforeLoad: () => {
    throw redirect({ to: '/inbox' });
  },
});
```

7. Live data from the running daemon

One read-only `GET /api/inbox` (no filters, so every project and every state) against `127.0.0.1:4317` on `2026-09-29` returned **113** items, created between `2026-09-22T14:47:42.983Z` and `2026-09-29T12:56:35.851Z`. Only counts are shown here. No item was `snoozed` or `done` at the time of the call.



kind	open	snoozed	done	auto_resolved	total
pr_event	21	0	0	6	27
review	3	0	0	15	18
waiting	2	0	0	66	68
total	26	0	0	87	113

- The screenshot in §1 is scoped to one project (Wakecap), so its tab counts (Open 17, Done 51) are lower than the unscoped totals above.
- Only three of the eleven kinds appear in this data: `waiting`, `review` and `pr_event`.
- The open `pr_event` rows in the screenshot read "merged; worktree kept because it was not created by the app", which is the `archive_blocked` facet from `daemon/services/worktree/auto-archive.ts:20-30`. The rows marked `×4` are four items with the same kind, reason and ticket, grouped by `groupItems`.

8. What the code does not settle

- **blocked kind**. It is in the type, the schema, the UI labels and the default notification preferences, but nothing in `apps/daemon/src` creates one. Whether it is planned or left over can't be told from the code.
- **"Permission" label**. The UI relabels a `waiting` item whose `payload.tool` is a string [`kinds.ts:71-73`]. The status rule's payload is `{ source, id, status }` [`status-rules.ts:73`], and no daemon inbox producer sets `tool`. It can't be told from the code when this label ever shows.
- **payload.approveText**. The approve service reads it for plan items when the plan service is absent [`services/remote/session-actions.ts:109-112`]; no producer writes it.
- **supervisor_escalation and reminder** have no automatic resolve path; they appear to leave only by user action (*inferred from grep for `resolve` calls*).
- **Plan items and Open/Enter**. Plan-approval payloads carry no `source/id`, so the row has no Open button and Enter does nothing (§2 note). Whether that is intended can't be told from the code.
- **Shared query between shell and page**. `AppShell` asks for `{state:['open']}` and the page asks for `{state:['open'], projectId: undefined}`. TanStack Query hashes keys through JSON, which drops `undefined`, so the two probably share one cache entry (*inferred; library behaviour, not repo code*).
- **Remote step-up rules** for `/api/inbox/:id/approve` live in the remote guard (`REMOTE_RULES`, referenced at `daemon/http/routes/session-actions.ts:12-13`) and were not traced further here.